大模型训练 LM Head 全解：前向、反向、FLOPs、显存与优化

> **原始问题：**详细讲解大模型训练中lm head这部分，包括前反向的每一步。举一个简单例子说明清楚，给出每一步的shape，计算公式，flops，作用和显存开销等大模型训练关心的数据。配合图片辅助我进行理解。我当前在做一个lm head显存优化的工作，讲完上述内容后再重点关注这部分变化，帮我画出一个动态显存开销图，在网页上清晰地展现出来每一步的计算步骤，计算量和显存开销变化。

提问：2026/08/27 10:35　更新：2026/08/27 10:51　重要度：5/5

大模型训练 LM Head 全解：前向、反向、FLOPs、显存与优化

一句话结论

LM Head（Language Modeling Head）是把 Transformer 最后一层每个 Token 的隐藏向量 `[H]` 投影成全词表分数 `[V]` 的输出层。 训练时它通常紧接稳定的 Softmax Cross Entropy：

```
Final hidden states H_final [B,S,H]
→ reshape 为 X [N,H]
→ Vocabulary projection: Z = XW_vocab^T
→ Logits Z [N,V]
→ Cross Entropy with target token ids [N]
→ scalar Loss
```

其中：

```
B  Micro Batch Size
S  Sequence Length
N  参与 Loss 的有效 Token 数, N ≤ B×S
H  Hidden Size
V  Vocabulary Size
W_vocab [V,H]
```

LM Head 训练最关键的系统事实是：

1. **计算量大**：前向矩阵乘约 `2NHV FLOPs`；连同 `dH` 和 `dW` 两个反向 GEMM，标准训练主 GEMM 总量约 `6NHV FLOPs`。
2. **Logits 显存可能巨大**：完整 Logits 有 `N×V` 个元素，和 Hidden Size 无关；长序列、大 Batch、大词表时，它可能比最后层 Hidden 大几十倍。
3. **Cross Entropy 并不需要长期保存完整 Logits**：每个 Token 的 Loss 只需要目标类别 Logit 和所有类别的 `logsumexp`；通过 Vocabulary Parallel、在线 Softmax、Chunk、Fusion 和重计算，可以避免完整 `[N,V]` Tensor 长时间驻留。
4. **显存优化不是免费午餐**：分块减小峰值，却可能增加重计算、Kernel Launch、通信、Buffer 管理和尾部低利用率。
5. **你当前做 LM Head 显存优化时，应优先定位峰值到底来自哪一份大 Tensor**：Logits、保存的 Softmax/Log-probs、`dLogits`、Gather 后的全词表 Logits、还是框架额外副本。

动态分析网页：

```
https://psgmessi.github.io/ai-infra-learning/lm-head-memory-explorer.html
```

网页支持动态调整 `B/S/H/V/dtype/TP/Token Chunk/Vocab Chunk`，逐阶段查看 Shape、FLOPs、显存峰值和实现方案之间的变化。

一、LM Head 在大模型中的位置

以 Decoder-only Transformer 为例：

```
Token IDs [B,S]
→ Token Embedding [B,S,H]
→ L 个 Transformer Blocks
→ Final RMSNorm / LayerNorm
→ H_final [B,S,H]
→ LM Head
→ Logits [B,S,V]
→ Shifted Cross Entropy
→ Loss scalar
```

LM Head 本身通常就是一个无 Bias 的 Linear：

```
logits[b,s,v]
= Σ_h H_final[b,s,h] × W_vocab[v,h]
```

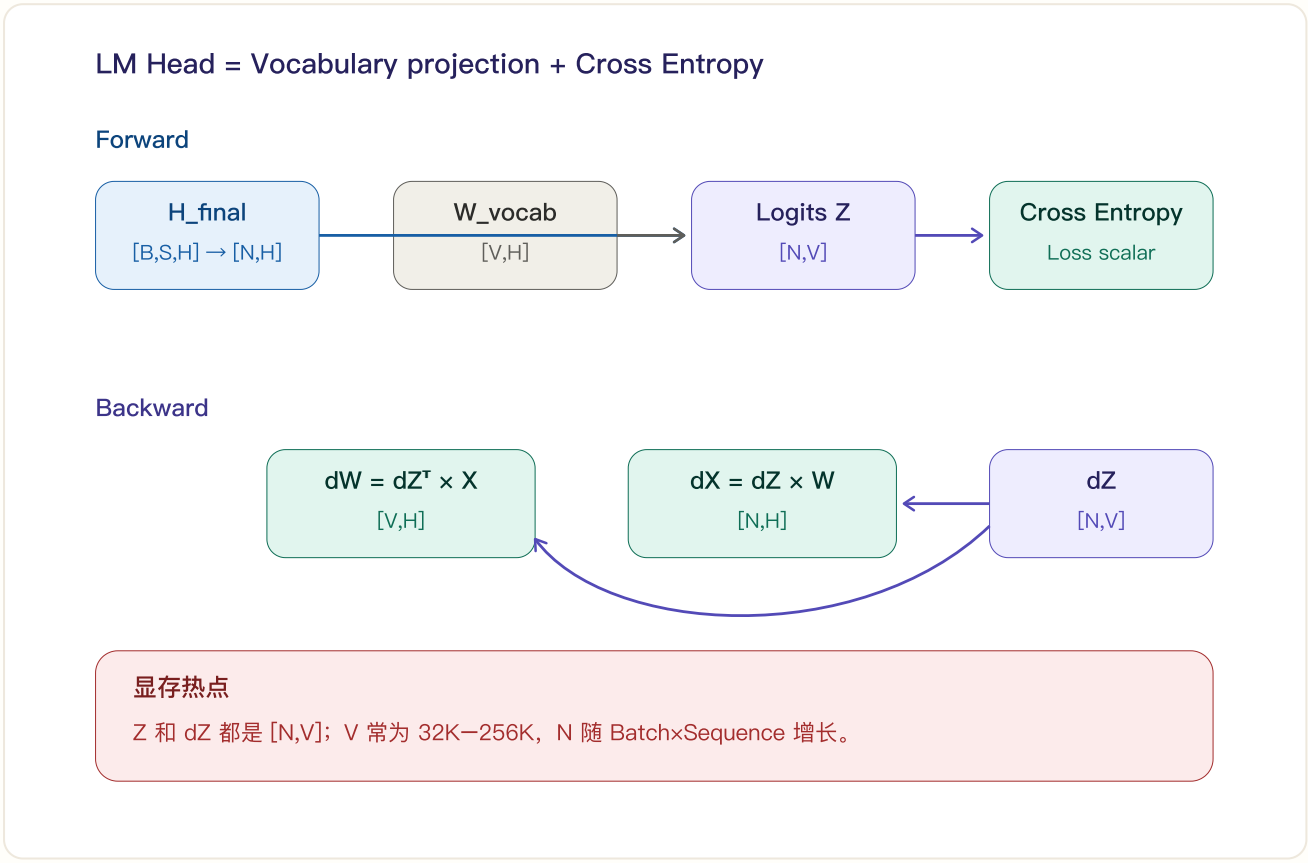
矩阵写法：

```
X = reshape(H_final, [N,H])
```

```
Z = X × W_vocab^T
```

Shape：

```
X          [N,H]
W_vocab    [V,H]
W_vocab^T  [H,V]
Z          [N,V]
```



LM Head 的作用

LM Head 把每个位置的上下文表示转换成：

```
对词表中每个 Token 的未归一化分数
```

注意它输出的是 Logits，不是概率：

```
Z[n,v] ∈ (−∞,+∞)
```

概率由 Softmax 得到：

```
P[n,v] = exp(Z[n,v]) ÷ Σ_j exp(Z[n,j])
```

训练目标是让目标 Token 的概率增加，非目标 Token 的总概率降低。

二、Causal LM 的 Label Shift 与有效 Token 数

Decoder-only Causal LM 使用当前位置的 Hidden State 预测下一个 Token。

假设 Token IDs：

```
[t0, t1, t2, t3]
```

训练配对：

```
位置 0 的 Hidden → 预测 t1
位置 1 的 Hidden → 预测 t2
位置 2 的 Hidden → 预测 t3
位置 3 没有下一个 Token，通常忽略
```

因此代码常见：

```
shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]
```

实际有效 Token 数不是机械的 `B×S`：

```
N_valid
= 非 Padding
- Ignore Index
- Sequence 边界不参与预测的位置
- 被 Loss Mask 排除的位置
```

性能估算必须明确使用：

```
N = 有效或实际进入 LM Head/CE 的 Token 数
```

有些实现先对全部 `B×S` 位置做 LM Head，再在 Loss 里 Mask；这种情况下投影计算量仍按 `B×S`，即使有效 Loss Token 更少。若能在投影前 Compact 有效 Token，才会真正减少 `N×V` 计算与显存，但会引入 Gather/Scatter、动态 Shape 和数据布局成本。

三、Weight Tying：LM Head 和 Embedding 是否共用参数

很多语言模型使用权重共享：

```
Input Embedding Weight E [V,H]
LM Head Weight W_vocab [V,H]

W_vocab 与 E 指向同一参数
```

优点：

- 少存一份 `[V,H]` 参数；
- 少存一份参数梯度和优化器状态；
- 输入和输出 Token 表示共享几何空间。

但反向时同一个参数有两条梯度来源：

```
dW_total
= dW_from_lm_head
+ dW_from_input_embedding
```

这不是覆盖，而是累加。

对于输入 Embedding，只有本 Batch 出现过的 Token 行有直接 Embedding Lookup 梯度；LM Head 的 `dW_vocab` 通常是 Dense 的，因为每个词表类别都参与 Softmax 分母。

如果模型使用 Untied LM Head，则参数状态额外增加：

```
参数：V×H×param_bytes
梯度：V×H×grad_bytes
Adam m/v：V×H×8 Byte（FP32）
FP32 Master Weight：可再加 V×H×4 Byte
```

四、统一小例子：2 个 Token、Hidden=3、Vocab=4

设置：

```
N = 2
H = 3
V = 4
```

最后层 Hidden：

```
X [N,H]
= [[1.0, 0.0, -1.0],
   [0.5, 1.0,  0.0]]
```

LM Head 权重：

```
W [V,H]
= [[1.0,  0.0, 0.0],    # Token 0
   [0.0,  1.0, 0.0],    # Token 1
   [0.0,  0.0, 1.0],    # Token 2
   [1.0, -1.0, 0.5]]    # Token 3
```

目标 Token：

```
y [N] = [0,2]
```

含义：

- 第一个位置应该预测 Token 0；
- 第二个位置应该预测 Token 2。

五、LM Head 前向第 1 步：Vocabulary Projection

公式：

```
Z = XW^T
```

Shape：

```
X    [2,3]
W^T  [3,4]
Z    [2,4]
```

第一个 Token Hidden：

```
x0 = [1,0,-1]
```

与四个词表向量做点积：

```
z0,0 = [1,0,-1]·[1,0,0]      = 1
z0,1 = [1,0,-1]·[0,1,0]      = 0
```

```
z0,2 = [1,0,-1]·[0,0,1]      = -1
z0,3 = [1,0,-1]·[1,-1,0.5]   = 0.5
```

第二个 Token：

```
x1 = [0.5,1,0]
```

```
z1,0 = 0.5
z1,1 = 1.0
z1,2 = 0.0
z1,3 = 0.5-1 = -0.5
```

所以：

```
Z [2,4]
= [[1.0, 0.0, -1.0,  0.5],
   [0.5, 1.0,  0.0, -0.5]]
```

Projection FLOPs

一个长度为 `H` 的 Dot Product：

```
H 次乘法 + H-1 次加法
≈ 2H FLOPs
```

共有 `N×V` 个 Dot Products：

```
FLOPs_forward_projection
≈ 2NHV
```

本例：

```
2×2×3×4 = 48 FLOPs
```

这里遵循常见口径：一次乘法和一次加法分别计 1 FLOP；Tensor Core FMA 常按 2 FLOPs 计。

六、LM Head 前向第 2 步：稳定 Softmax Cross Entropy

不能直接计算：

```
exp(z_v) ÷ Σ_j exp(z_j)
```

因为大 Logit 的 `exp` 可能溢出。稳定算法先减最大值。

对每个 Token `n`：

```
m_n = max_v Z[n,v]

s_n = Σ_v exp(Z[n,v]-m_n)

logsumexp_n = m_n + log(s_n)

loss_n = logsumexp_n - Z[n,y_n]
```

这与直接 Softmax 完全等价，但数值更稳定。

第一个 Token

```
z0 = [1,0,-1,0.5]
m0 = 1
```

减最大值：

```
[0,-1,-2,-0.5]
```

指数：

```
[1,
 exp(-1),
 exp(-2),
 exp(-0.5)]

≈ [1,0.367879,0.135335,0.606531]
```

求和：

```
s0 ≈ 2.109745
```

概率：

```
P0
≈ [0.473991,
   0.174371,
   0.064148,
   0.287490]
```

目标是 Token 0：

```
loss0 = -log(P0[0])
      ≈ 0.746567
```

第二个 Token

```
z1 = [0.5,1,0,-0.5]
m1 = 1
```

概率：

```
P1
≈ [0.276004,
   0.455054,
   0.167405,
   0.101536]
```

目标为 Token 2：

```
loss1 = -log(P1[2])
      ≈ 1.787339
```

Mean Loss：

```
L = (loss0+loss1) ÷ 2
  ≈ 1.266953
```

Cross Entropy 计算量

每个 Logit 大致经历：

```
比较 max
减法
exp
sum
归一化或 logsumexp
```

可用线性量级表示：

```
FLOPs_CE ≈ O(NV)
```

若把比较、减法、加法等粗略按 4–6 次标量操作计：

```
≈ 4NV 到 6NV 标量操作 + N 次 log
```

对于大 `H`，`2NHV` 的投影 GEMM 通常在 FLOPs 上远高于 `O(NV)`；但 Cross Entropy 的 Exp、Reduce 和大 Tensor HBM 读写仍可能成为显著性能与显存瓶颈。

七、Cross Entropy 反向：为什么 dLogits = P − OneHot

对单个 Token：

```
loss = log(Σ_j exp(z_j)) − z_y
```

对任意类别 `k`：

```
∂loss/∂z_k
= exp(z_k) ÷ Σ_j exp(z_j) − 1[k=y]

= P_k − 1[k=y]
```

若 Loss 对 `N` 个 Token 取 Mean：

```
dZ[n,v]
= (P[n,v] − 1[v=y_n]) ÷ N
```

本例 `N=2`。

第一个 Token，目标为 0

```
P0 − one_hot(0)
= [0.473991−1,
   0.174371,
   0.064148,
   0.287490]

= [−0.526009,
    0.174371,
    0.064148,
    0.287490]
```

除以 2：

```
dZ0
≈ [−0.263005,
    0.087186,
    0.032074,
    0.143745]
```

第二个 Token，目标为 2

```
dZ1
≈ [ 0.138002,
    0.227527,
   −0.416297,
    0.050768]
```

最终：

```
dZ [N,V]
≈ [[-0.263005, 0.087186, 0.032074, 0.143745],
   [ 0.138002, 0.227527,-0.416297, 0.050768]]
```

解释：

- 目标类别梯度为负，梯度下降会提高目标 Logit；
- 非目标类别梯度为正，梯度下降会降低这些 Logits；
- 每行梯度之和接近 0，因为 Softmax 对所有 Logits 同时加常数不敏感。

检查：

```
Σ_v dZ[n,v] ≈ 0
```

八、LM Head Linear Backward：dHidden 和 dWeight

前向：

```
Z = XW^T
```

收到：

```
dZ [N,V]
```

8.1 对隐藏状态的梯度

```
dX = dZ × W
```

Shape：

```
dZ [N,V]
W  [V,H]
dX [N,H]
```

本例：

```
dX
≈ [[-0.119260,-0.056559, 0.103946],
```

```
   [ 0.188770, 0.176759,-0.390913]]
```

这个 `dX` 会继续传给 Final Norm 和最后一个 Transformer Block。

FLOPs：

```
FLOPs_dX ≈ 2NVH
```

8.2 对 LM Head 权重的梯度

```
dW = dZ^T × X
```

Shape：

```
dZ^T [V,N]
X    [N,H]
dW   [V,H]
```

本例：

```
dW
≈ [[-0.194003, 0.138002, 0.263005],
   [ 0.200949, 0.227527,-0.087186],
   [-0.176075,-0.416297,-0.032074],
   [ 0.169129, 0.050768,-0.143745]]
```

FLOPs：

```
FLOPs_dW ≈ 2NVH
```

8.3 LM Head 训练总主 GEMM FLOPs

```
Forward Projection ≈ 2NHV
dHidden            ≈ 2NHV
dWeight            ≈ 2NHV
```

合计：

```
FLOPs_LMHead_GEMM_train
≈ 6NHV
```

再加：

```
Softmax / Cross Entropy ≈ O(NV)
```

如果使用不保存完整 Logits、Backward 重算 Projection 的 Fused Linear CE，则通常还要增加一次近似：

```
Recompute Logits ≈ 2NHV
```

保守主 GEMM 总量变成：

```
≈ 8NHV
```

相对标准 `6NHV`，理论上增加约：

```
(8−6) ÷ 6 = 33.3%
```

注意这是 **LM Head 子模块内部** 的增量，不代表整个模型 Step Time 增加 33%。如果 LM Head 只占全模型计算的一小部分，端到端增量会更小；如果长序列、大词表让 LM Head 占比很高，影响会更明显。

九、真实规模下的 FLOPs 与显存

使用一个可复算配置：

```
B = 2
S = 8192
N = B×S = 16384
H = 4096
V = 128256
Logits dtype = BF16 = 2 Byte
```

9.1 完整 Logits

元素数：

```
N×V
= 16384×128256
= 2,101,346,304 elements
```

显存：

```
2,101,346,304 × 2 Byte
= 4,202,692,608 Byte
≈ 3.914 GiB
```

一份 BF16 Logits 就接近 `3.914 GiB`。

若 Backward 峰值同时存在一份同 Shape 的 `dLogits`：

```
Logits + dLogits
≈ 7.828 GiB
```

这还没有包括：

- Softmax/LogSoftmax 保存结果；
- Cross Entropy Workspace；
- Hidden States；
- LM Head Weight 和 `dW`；
- cuBLAS/Triton Workspace；
- CUDA Allocator 碎片；
- TP Gather Buffer；
- 模型其他层。

具体实现可能原地复用 Logits Buffer，因此“同时两份”是一个显式 Tensor 上界模型，而不是所有 Kernel 的固定事实。

9.2 Hidden States

```
N×H×2 Byte
= 16384×4096×2
= 134,217,728 Byte
= 128 MiB
```

对比：

```
Logits / Hidden
= V / H
= 128256 / 4096
≈ 31.31
```

因此同 dtype 下，完整 Logits 是最后层 Hidden 的约 `31.3 倍`。

9.3 LM Head Weight

```
V×H×2 Byte
= 128256×4096×2
≈ 0.979 GiB
```

`dW` 若为 BF16 同样约 `0.979 GiB`；若梯度或参数副本使用 FP32，则翻倍。

9.4 FLOPs

Forward Projection：

```
2NHV
= 2×16384×4096×128256
≈ 17.21 TFLOPs
```

标准训练三个 GEMM：

```
6NHV
≈ 51.64 TFLOPs
```

这是每个 Micro-batch 的算法 FLOPs，不是 FLOP/s，也没有包含 Final Norm、CE 标量操作或通信。

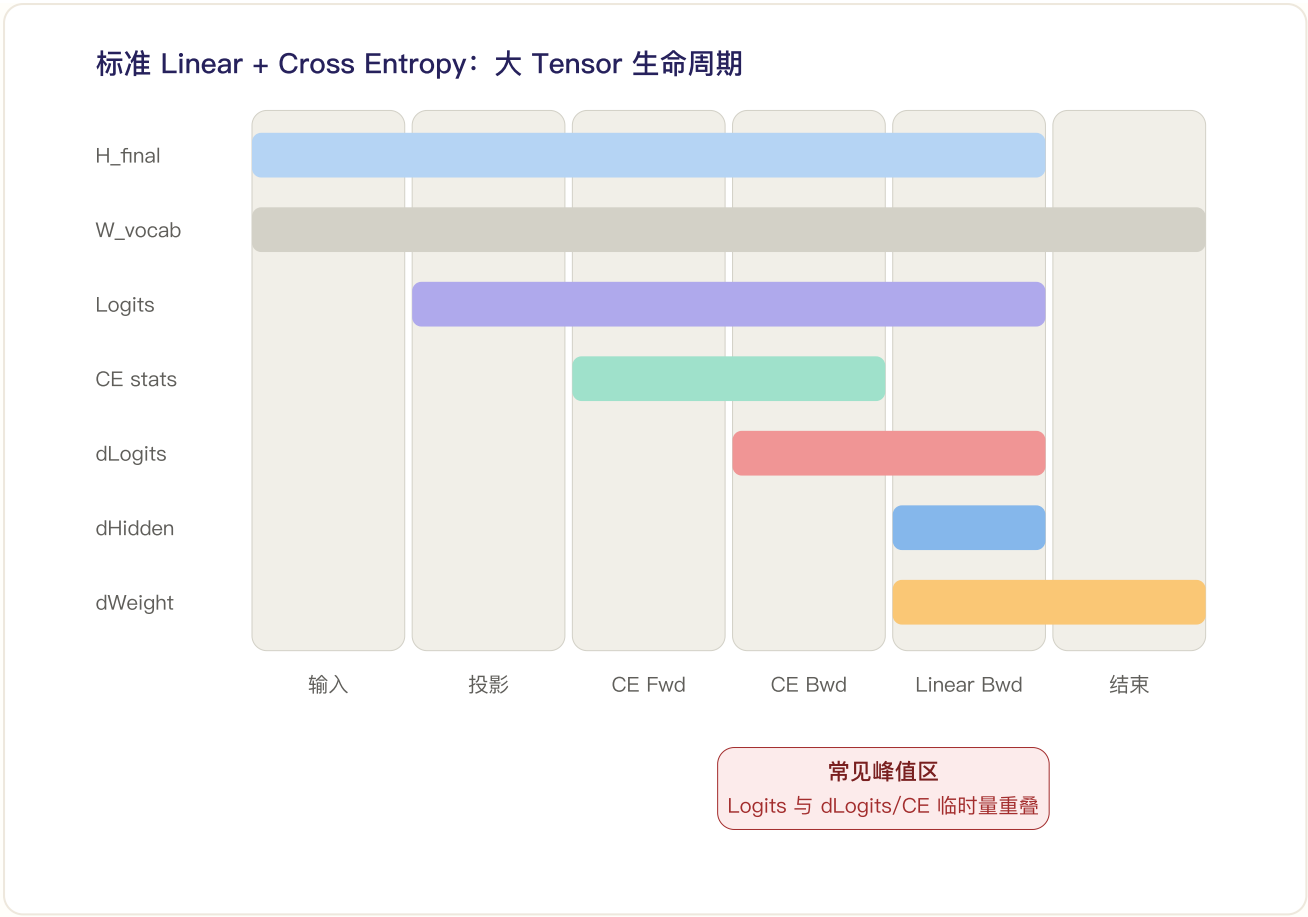
十、标准实现的动态显存时间线

标准路径：

```
logits = hidden @ weight.T
loss = cross_entropy(logits, labels)
loss.backward()
```

一个保守的生命周期模型：

```
阶段 1: H_final 和 W 已存在
阶段 2: 生成完整 Logits [N,V]
阶段 3: CE 计算/保存 Softmax 所需信息
阶段 4: 产生 dLogits [N,V]
阶段 5: 用 dLogits 计算 dH 和 dW
阶段 6: 释放 Logits/dLogits，保留参数梯度
```



为什么单看 `torch.cuda.memory_allocated()` 可能误判

需要区分：

```
allocated: Tensor 实际占用
reserved: CUDA Caching Allocator 向驱动保留
peak allocated: 历史最大 Tensor 占用
workspace: 算子临时空间
fragmentation: 空闲块无法满足当前分配
```

LM Head 优化后，`allocated` 峰值可能下降，但 `reserved` 不一定立即下降；反过来，Kernel 内部 Workspace 可能让实际峰值高于手算 Tensor 总和。

十一、显存优化路线 1：降低 Logits 精度

若 Logits 从 FP32 改为 BF16：

```
[N,V] × 4 Byte
→ [N,V] × 2 Byte
```

显存减半。

但稳定 Softmax 通常仍建议：

```
输入 Logits：BF16/FP16
max / sum-exp accumulation：FP32
Loss：FP32
```

风险：

- FP16 指数范围小；
- 大词表下极小概率更容易下溢；
- Label Smoothing、Logit Soft-Capping 等会改变数值路径；
- FP8 Logits 需要专门的缩放和误差验证，不能只按 1 Byte 容量推断可用性。

这是低风险的容量优化，但无法消除 `N×V` 规模本身。

十二、显存优化路线 2：Vocabulary Parallel LM Head

令 Tensor Parallel Degree 为 `P`，沿 Vocabulary 维切分：

```
W_vocab [V,H]
→ 每 Rank W_r [V/P,H]
```

每 Rank 只产生：

```
Z_r [N,V/P]
```

参数、`dW` 和 Local Logits 约缩小到 `1/P`。

12.1 Distributed Stable Softmax

每 Rank 先算 Local Max：

```
m_r[n] = max_{v in local shard} Z_r[n,v]
```

跨 TP 做 MAX AllReduce：

```
m[n] = max_r m_r[n]
```

各 Rank 计算：

```
s_r[n]
= Σ_{v in local shard} exp(Z_r[n,v]-m[n])
```

跨 TP 做 SUM AllReduce：

```
s[n] = Σ_r s_r[n]
```

只有拥有目标 Token 的 Rank 取出 Target Logit；其他 Rank 填 0：

```
t_r[n] = local target logit or 0
```

SUM AllReduce 得到全局 Target Logit：

```
t[n] = Σ_r t_r[n]
```

Loss：

```
loss[n]
= m[n] + log(s[n]) - t[n]
```

Megatron Core 的 `VocabParallelCrossEntropy` 就采用这类路径：全局 Max、Target Logit SUM、Sum-Exp SUM，并把 Local Exp Logits 原地归一化为 Local Softmax 保存给 Backward。

12.2 TP Backward

每 Rank 只生成 Local `dZ_r [N,V/P]`：

```
dZ_r = P_r - local_one_hot
```

Local Weight Gradient：

```
dW_r = dZ_r^T X
```

Local Hidden Gradient Contribution：

```
dX_r = dZ_r W_r
```

全局 `dX`：

```
dX = Σ_r dX_r
```

因此通常需要 TP SUM AllReduce，或与 Sequence Parallel 组合成 ReduceScatter。

12.3 通信量

Forward 的统计通信主要是长度 `N` 的 FP32 向量：

```
Global Max
Global Sum-Exp
Target Logit
```

量级约：

```
3 × N × 4 Byte
```

Backward 的 `dX` 通信量级：

```
N × H × grad_bytes
```

通常比 Forward 标量统计大得多。

12.4 适用条件

适合：

- 已使用 Tensor Parallel；
- `V` 很大；
- LM Head Weight 本身也需要分片；
- 网络拓扑足够快。

代价：

- 多次 TP Collective；
- 最慢 Rank 决定同步完成时间；
- 若错误地 AllGather 全词表 Logits，会重新引入 `[N,V]` 峰值，抵消分片收益。

十三、显存优化路线 3：Token Chunk

把 `N` 个 Token 分成大小 `C_t` 的块：

```
X [N,H]
→ X_0 [C_t,H], X_1 [C_t,H], ...
```

每块：

```
Z_i = X_i W^T        [C_t,V]
loss_i = CE(Z_i,y_i)
```

瞬时 Logits 从：

```
N×V
```

降到：

```
C_t×V
```

真实配置：

```
C_t = 256
V = 128256
BF16
```

Chunk Logits：

```
256×128256×2 Byte
≈ 62.6 MiB
```

相比完整 `3.914 GiB`，单块明显更小。

关键陷阱：普通 Python Loop 不一定真正省 Backward 峰值

如果只是：

```
loss = 0
for chunk in chunks:
    logits = chunk @ W.T
    loss = loss + cross_entropy(logits, labels_chunk)
loss.backward()
```

Autograd 可能为每个 Chunk 保存一张子图，直到最后统一 `backward()`。所有 Chunk 的 Saved Tensors 仍可能累计，不能仅凭“循环里覆盖了 Python 变量 logits”就断言显存变成一个 Chunk。

真正释放通常需要：

- 自定义 `autograd.Function`；
- Forward 只保存小统计量，Backward 重算 Chunk；
- 或逐 Chunk Backward 并正确缩放/累加梯度；
- 或框架提供的 Fused Linear Cross Entropy。

十四、显存优化路线 4：Vocabulary Chunk + Online Softmax

把 Vocabulary 分块：

```
W [V,H]
→ W_0 [C_v,H], W_1 [C_v,H], ...
```

每次只产生：

```
Z_chunk [N,C_v]
```

问题是 Softmax 分母需要全词表：

```
Σ_v exp(z_v)
```

可以使用 Online Softmax 合并每块统计量。

假设当前已经处理一部分 Vocabulary，维护：

```
m_old = 当前全局最大值
l_old = Σ exp(z-m_old)
```

新 Chunk 的统计：

```
m_chunk = max(z_chunk)
l_chunk = Σ exp(z_chunk-m_chunk)
```

合并：

```
m_new = max(m_old,m_chunk)

l_new
= l_old × exp(m_old-m_new)
+ l_chunk × exp(m_chunk-m_new)
```

处理完全部 Vocabulary 后：

```
logsumexp = m_final + log(l_final)
```

同时记录目标 Token 所在 Chunk 的 Target Logit，就能精确计算 Cross Entropy，而无需完整 `[N,V]` Logits。

显存从：

```
N×V×bytes
```

降到：

```
N×C_v×bytes
+ O(N) FP32 statistics
```

代价：

- 多个小 GEMM；
- Chunk 太小会降低 Tensor Core 利用率；
- 更多 Kernel Launch；
- Backward 通常需要重算 Chunk Logits。

十五、显存优化路线 5：Fused Linear Cross Entropy

标准边界：

```
Linear 输出完整 Logits
→ Framework 保存 Logits/Softmax
→ Cross Entropy
→ Backward 生成完整 dLogits
→ Linear Backward
```

Fusion 后：

```
linear_cross_entropy(hidden, weight, labels)
```

把 Projection 和 CE 视为一个整体算子。

Forward

分 Chunk/Tile 执行：

```
1. 计算一小块 Logits
2. 更新 per-token max 和 sum-exp
3. 如果目标 Token 在当前块，保存 Target Logit
```

```
4. 立即覆盖或释放当前 Logits Tile
5. 最终得到 Loss
```

只需长期保存：

```
Hidden [N,H]
Weight [V,H]
Target [N]
Max [N] FP32
SumExp [N] FP32
```

Backward

为了计算：

```
dZ = softmax(Z) − one_hot
```

如果 Forward 没保存完整 Z，就必须：

```
重新按 Chunk 计算 Logits
→ 使用保存的 max/sum-exp 恢复概率
→ 生成一个 dZ Chunk
→ 立即累计 dH 和 dW
→ 释放 dZ Chunk
```

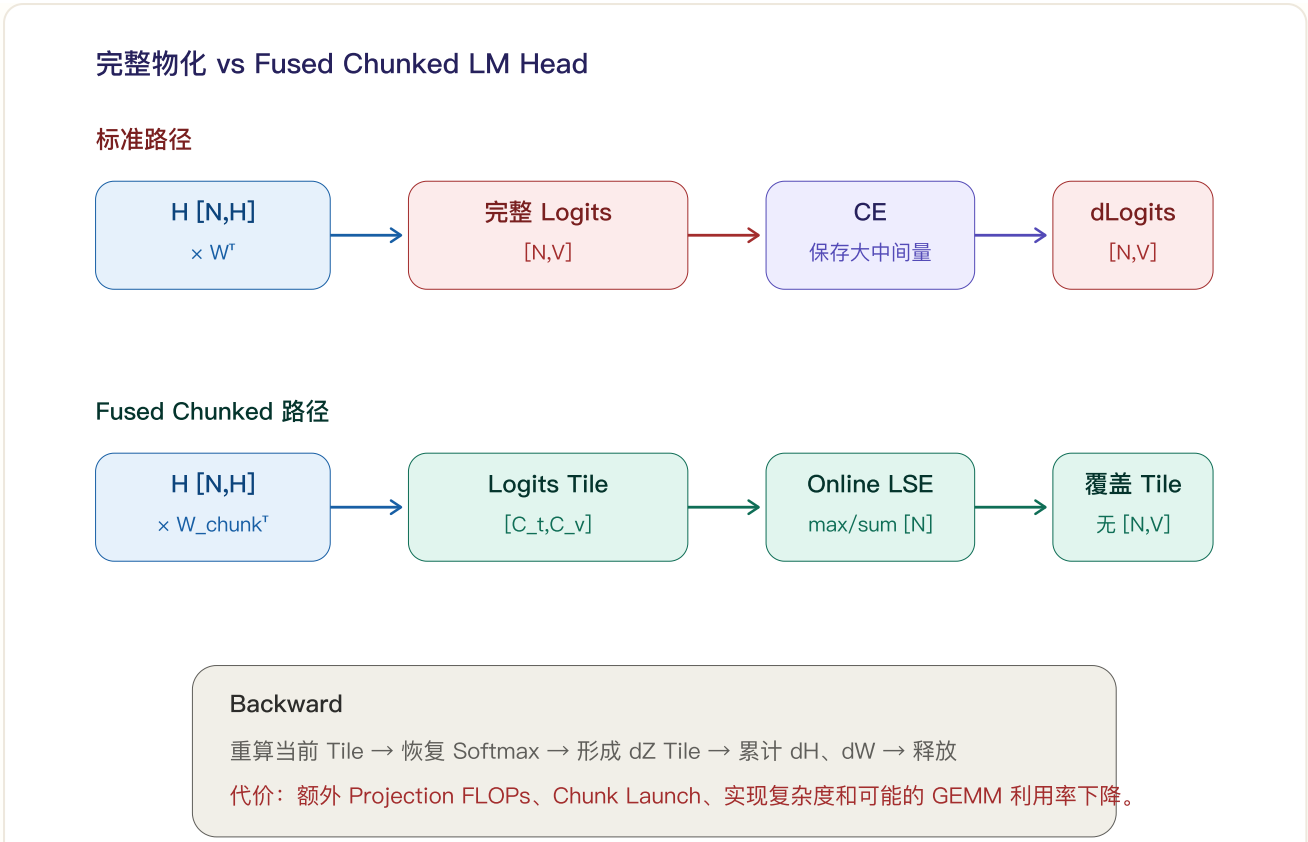
峰值由：

```
Token Chunk × Vocab Chunk
```

而不是：

```
N × V
```

决定。



一个重要的实现分歧

不同 Fused Linear CE 可能采用不同策略：

1. **Forward 只保存统计，Backward 重算**：低峰值、增加约一次 Projection；
2. **Forward 直接预计算部分梯度**：减少 Backward 重算，但需保存 `dH/dW` 或特殊状态；
3. **Token Chunk**：Tile `[C_t,V]`；
4. **Vocab Chunk**：Tile `[N,C_v]`；
5. **二维 Tile**：Tile `[C_t,C_v]`；
6. **TP + Chunk**：每 Rank 在 `V/P` 内继续分块。

不能只看到“Fused Linear CE”名字就假设显存、FLOPs 和通信完全一样，必须看具体实现。

十六、Cut Cross Entropy（CCE）

Cut Cross Entropy 的核心目标同样是：

```
不把所有 Token × Vocabulary Logits 写入全局显存
```

其核心思路：

- 只显式关注正确类别 Logit；
- 对所有类别的 `logsumexp` 在 Tile 中在线归约；
- 自定义 Kernel 把矩阵乘和 LSE Reduction 紧密结合；
- 中间 Logits 尽量停留在片上存储或短生命周期 Tile 中；
- Backward 可利用 Softmax 梯度的数值稀疏性，跳过低于数值精度、贡献可忽略的项，以改善吞吐。

Apple CCE 工作报告：在其 Gemma 2 2B 配置中，Loss 计算内存从 24 GB 降至约 1 MB，Classifier Head 总训练内存从 28 GB 降至约 1 GB。**这些是特定模型、实现与测量口径的报告结果，不能直接外推到你的模型。**

CCE 与普通 Chunk 的关键差异不只是“写一个 for 循环”，而是：

```
矩阵乘 Tile
+ Online LSE
+ Target Gather
+ Backward Gradient Tile
```

在定制 Kernel 中协同调度，避免全局显存往返和 Autograd 子图堆积。

十七、优化方案的计算—显存—通信对比

方案	完整 Logits	每 Rank 权重	主 GEMM FLOPs	通信	主要风险
标准 Linear+CE	`[N,V]`	`[V,H]`	约 `6NHV`	DP 梯度同步	峰值大
BF16 Logits	`[N,V]`	`[V,H]`	近似不变	不变	仍是 `N×V`
Vocab TP=P	`[N,V/P]` Local	`[V/P,H]`	每 Rank约 `6NHV/P`	Max/Sum/Target + dH AllReduce	网络与同步
Token Chunk	`[C_t,V]`	`[V,H]`	若重算约 `8NHV`	通常不新增	小 GEMM/Autograd 图陷阱
Vocab Chunk	`[N,C_v]`	`[V,H]`	若重算约 `8NHV`	通常不新增	多次扫 Hidden/Weight
TP + Chunk	`[C_t,C_v]`	`[V/P,H]`	每 Rank约 `8NHV/P`	TP Collectives	实现复杂

方案	完整 Logits	每 Rank 权重	主 GEMM FLOPs	通信	主要风险
CCE-like	Tile，不全局物化	`[V,H]` 或分片	上界约 `8NHV`，可有稀疏跳过	依并行实现	专用 Kernel 与数值验证

结论翻转条件

- `V/H` 很小、序列很短：完整 Logits 未必是峰值，Fusion 收益有限；
- `V` 很大、`N` 很大：LM Head 优化收益显著；
- GPU 算力强但 HBM 紧张：重算换显存通常更有吸引力；
- 网络慢：Vocab TP 可能省显存但通信暴露变大；
- Chunk 太小：显存继续下降，但 Tensor Core 利用率和 Launch 效率恶化；
- Weight Tied：参数显存已共享，但 Logits 动态显存仍然存在；
- Sequence Parallel/Context Parallel：每 Rank `N` 可能下降，但最后是否 Gather、如何计算 Loss 决定真实峰值。

十八、针对你当前 LM Head 显存优化工作的分析框架

18.1 第一件事：确认峰值 Tensor 身份

不要先猜方案，先记录峰值前后的大分配：

```
H_final             [N,H]
LM Head Weight      [V,H] 或 [V/P,H]
Logits              [N,V] 或 [N,V/P]
Saved Softmax       [N,V] 或 Local
Logits Gradient     [N,V] 或 Local
Weight Gradient     [V,H] 或 Local
Hidden Gradient     [N,H]
Gather/Reduce Buffer
Kernel Workspace
```

推荐同时看：

```
torch.cuda.reset_peak_memory_stats()
# run target region
torch.cuda.synchronize()
print(torch.cuda.max_memory_allocated() / 2**30)
print(torch.cuda.max_memory_reserved() / 2**30)
```

以及：

```
torch.cuda.memory._record_memory_history()
# run step
torch.cuda.memory._dump_snapshot("lm_head_snapshot.pickle")
```

再结合 Nsight Systems 确认 Kernel 和 Collective 时间线。

18.2 建立分阶段 NVTX 标记

```
with torch.cuda.nvtx.range("final_norm"):
    hidden = final_norm(hidden)

with torch.cuda.nvtx.range("lm_head_projection"):
    logits = lm_head(hidden)

with torch.cuda.nvtx.range("cross_entropy"):
    loss = cross_entropy(logits, labels)

with torch.cuda.nvtx.range("backward"):
    loss.backward()
```

Fused 后继续细分：

```
fwd_chunk_projection
fwd_online_lse
bwd_recompute_projection
bwd_dhidden
bwd_dweight
TP stats allreduce
dHidden allreduce/reduce-scatter
```

18.3 必须同时记录的指标

显存

```
Peak allocated / reserved
Logits Tensor bytes
Saved Tensor bytes
dLogits bytes
Workspace bytes
Allocator fragmentation
每阶段生命周期
```

计算

```
Projection GEMM duration
CE duration
Backward recompute duration
dH GEMM duration
dW GEMM duration
Tensor Core utilization
Achieved TFLOP/s
```

带宽

```
HBM read/write throughput
L2 hit rate
Weight 重读次数
Hidden 重读次数
```

通信

```
TP max/sum/target collectives
dH AllReduce/ReduceScatter
通信是否与其他计算重叠
Gather Buffer 是否完整物化全 Vocab
```

数值

```
Loss absolute/relative error
dHidden max/mean error
dWeight max/mean error
长训练 Loss Curve
NaN/Inf
不同 dtype 和极端 Logit 范围
Ignore Index / Label Smoothing
```

18.4 公平对比方法

比较 Baseline 与优化版本时固定：

```
同一 Hidden 和 Labels
同一 Weight
同一 Loss Reduction
同一 Ignore Index
同一 Label Smoothing
同一 dtype/accumulation dtype
同一 TP/SP/CP 配置
```

校验：

```
Forward Loss
Per-token Loss
LM Head dW
dHidden
Tied Embedding 总梯度
一个完整 Optimizer Step 后参数
```

性能要先 Warmup，再多次测量 P50/P95，避免把首轮编译、Autotune 或 CUDA Context 初始化计入。

十九、Chunk Size 应如何选择

Chunk Size 是典型的 U 型权衡。

Chunk 太大

- 峰值显存高；
- 可并发/流水机会少；
- 可能再次 OOM。

Chunk 太小

- GEMM M/N 维太小；
- Tensor Core 利用率下降；
- Kernel Launch 数量增加；
- Online Reduction 和索引开销占比提高；
- Weight/Hidden 可能被重复从 HBM 读取；
- TP Collective 过碎时网络效率下降。

建议实验矩阵：

```
Token Chunk: 64,128,256,512,1024
Vocab Chunk: 1024,2048,4096,8192,16384
```

每个点记录：

```
Peak Memory
Step Time
LM Head Time
Achieved TFLOP/s
HBM Bandwidth
数值误差
```

最后画：

```
x 轴：Peak Memory
y 轴：Step Time
```

选择 Pareto Frontier，而不是只选显存最低的配置。

二十、动态图网页如何使用

打开：

```
https://psgmessi.github.io/ai-infra-learning/lm-head-memory-explorer.html
```

可切换：

```
标准 Linear + CE
Vocabulary Parallel
Token Chunk Fused CE
Vocabulary Chunk Fused CE
TP + 双维 Chunk
CCE-like Tile Kernel
```

可动态修改：

```
B、S、有效 Token 比例、H、V、dtype、TP、Token Chunk、Vocab Chunk
```

网页会更新：

1. 有效 Token 数；
2. 每 Rank Forward FLOPs；
3. 每 Rank训练 FLOPs；
4. 逐阶段解析峰值；
5. Weight、dW、Hidden、Logits、dLogits、Stats、dHidden 和通信 Buffer 的生命周期；
6. 与标准实现相比的峰值变化；
7. 每一步的 Shape、FLOPs 和活跃 Tensor。

默认真实规模：

```
B=2, S=8192, H=4096, V=128256, BF16
```

也提供长上下文 Preset。网页使用解析模型，最终结论必须以真实代码 Profiling 为准。

二十一、最小 PyTorch 基线代码

```
import torch
import torch.nn.functional as F
```

```
B, S, H, V = 2, 8, 16, 32

torch.manual_seed(0)
hidden = torch.randn(B, S, H, device="cuda", dtype=torch.bfloat16, requires_grad=True)
weight = torch.randn(V, H, device="cuda", dtype=torch.bfloat16, requires_grad=True)
labels = torch.randint(0, V, (B, S), device="cuda")

# Causal LM shift
x = hidden[:, :-1, :].contiguous()       # [B,S-1,H]
y = labels[:, 1:].contiguous()          # [B,S-1]

logits = x @ weight.t()                  # [B,S-1,V]
loss = F.cross_entropy(
    logits.view(-1, V),
    y.reshape(-1),
    reduction="mean",
)

loss.backward()

print("logits:", logits.shape)
print("loss:", float(loss))
print("dHidden:", hidden.grad.shape)
print("dWeight:", weight.grad.shape)
```

一个容易忽略的问题

如果 Logits 在 Loss 之前被额外用于：

- Accuracy/Top-k Metric；
- Distillation；
- KL Loss；
- Logit Regularization；
- 多任务 Loss；
- 保存或日志；

那么 Fused Linear CE 不能简单丢弃完整 Logits，或者需要为这些消费者提供新的流式接口。优化前必须梳理所有消费者。

二十二、最小 Fused Chunk 伪代码

下面展示算法结构，不是生产 Kernel：

```
class LinearCrossEntropy(torch.autograd.Function):
    @staticmethod
    def forward(ctx, hidden, weight, target, chunk_size):
        # hidden [N,H], weight [V,H]
        # 只保存小统计，不保存完整 [N,V] logits
        max_value = full((N,), -inf, fp32)
        sum_exp = zeros((N,), fp32)
```

```
        target_logit = zeros((N,), fp32)

        for v_start in range(0, V, chunk_size):
            Wc = weight[v_start:v_end]          # [Cv,H]
            Zc = hidden @ Wc.T                  # [N,Cv]

            # Online logsumexp merge
            chunk_max = Zc.max(-1)
            new_max = maximum(max_value, chunk_max)
            sum_exp = (
                sum_exp * exp(max_value-new_max)
                + exp(Zc-new_max[:,None]).sum(-1)
            )
            max_value = new_max

            # 当前块包含目标 token 时提取 target logit
            gather_target_if_owned(...)

        loss = max_value + log(sum_exp) - target_logit
        ctx.save_for_backward(hidden, weight, target, max_value, sum_exp)
        return loss.mean()

    @staticmethod
    def backward(ctx, grad_output):
        # 重新分块计算 logits/softmax，不生成完整 [N,V]
        d_hidden = zeros_like(hidden)
        d_weight = zeros_like(weight)

        for each vocab_chunk:
            Zc = hidden @ Wc.T
            Pc = exp(Zc-max_value[:,None]) / sum_exp[:,None]
            dZc = Pc
            subtract_one_hot_if_target_owned(dZc)
            dZc *= grad_output / N

            d_hidden += dZc @ Wc
            d_weight[v_start:v_end] += dZc.T @ hidden

        return d_hidden, d_weight, None, None
```

生产实现需要进一步处理：

- BF16/FP16 输入与 FP32 Reduction；
- Label Smoothing；
- Ignore Index；
- Z-loss / Logit Soft-Capping；
- TP/SP/CP；
- Weight Tying；
- Autocast；
- Distributed Gradient Semantics；
- Stream 与 Buffer 生命周期；
- Kernel Fusion 和 Tile 调度；
- 非整除 Vocabulary Padding。

二十三、常见误区

误区 1：只有目标 Token 的 Logit 有用，所以其他 Logits 不用算

Cross Entropy 分母需要：

```
Σ_v exp(z_v)
```

因此所有类别都参与归一化。可以不把所有 Logits 写入全局显存，但通常仍需计算其对 `logsumexp` 的贡献。

误区 2：把 logits 变量覆盖，就释放了显存

Autograd 可能仍通过计算图持有底层 Tensor。Python 变量名被覆盖不等于 Tensor 生命周期结束。

误区 3：Token Chunk 循环一定把峰值降为一个 Chunk

若所有 Chunk 的计算图都保留到最后统一 Backward，Saved Tensors 仍可能累计。需要自定义 Backward、重计算或逐块 Backward。

误区 4：Vocabulary Parallel 不需要完整词表信息

它不需要 Gather 完整 Logits，但仍需要跨 Rank 合并 Global Max、Sum-Exp 和 Target Logit；Backward 还需合并 `dHidden`。

误区 5：Fused 一定更快

Fusion 通常省 HBM，但分块重算和小 GEMM 可能增加计算与 Launch。是否更快由 `N/H/V/Chunk/GPU/TP` 共同决定。

误区 6：Weight Tying 消除了 LM Head 显存问题

它减少参数、梯度和优化器状态副本，但不消除 `[N,V]` Logits 动态显存。

误区 7：只比较 Forward Loss 就够了

必须比较 `dHidden`、`dWeight`、Tied Embedding 总梯度和完整 Optimizer Step。很多错误只出现在 Backward、Mask、Reduction 或分布式路径。

二十四、建议你的优化工作按这个顺序推进

第 1 阶段：精确诊断

```
确认完整 Logits 是否实际物化
确认保存的是 Logits、Softmax 还是 LogProbs
确认 Backward 是否又分配完整 dLogits
确认 TP 是否误 AllGather 全词表
确认 Metrics/Distillation 是否持有 Logits
确认峰值是在 CE Forward、CE Backward、dW 还是通信阶段
```

第 2 阶段：建立正确性基线

```
Tiny FP64/FP32 Case
Forward Loss 对齐
Per-token Loss 对齐
dHidden 对齐
dWeight 对齐
Ignore Index 对齐
Label Smoothing 对齐
TP 1/2/4 对齐
```

第 3 阶段：逐层加入优化

推荐增量顺序：

```
1. BF16 Logits + FP32 Reduction
2. 避免全词表 AllGather，使用 Vocab Parallel CE
3. 自定义 Fused Linear CE，先做较大 Chunk
4. 扫 Token/Vocab Chunk Size
5. 融合 Online LSE、Target Gather 和 dZ
6. 优化 dH/dW 累加与 Buffer 复用
7. 再考虑 CCE-like 片上 Tile 与梯度过滤
```

每一步都做数值和峰值回归，避免同时改变太多变量。

第 4 阶段：端到端裁决

最终不是看 LM Head 单算子，而是：

```
Peak Memory 是否下降
是否能增加 Sequence 或 Micro Batch
Step Time 是否下降
MFU 是否变化
通信是否暴露
```

```
训练 Loss Curve 是否一致
Checkpoint/Resume 是否正确
```

如果省下的显存可以把 Micro Batch 从 1 提高到 2，即使 Fused LM Head 自身略慢，端到端吞吐也可能提高；反之，若模型本来不 OOM，过度切 Chunk 可能只带来额外开销。

二十五、关键总结

1. LM Head 前向是 `Z=XW^T`，输出 Shape `[N,V]`，Projection FLOPs 约 `2NHV`。
2. Stable Cross Entropy 对每个 Token 计算 Global Max、Sum-Exp 和 Target Logit；数学上需要所有类别的贡献，但不要求完整 Logits 长期驻留显存。
3. CE Backward：`dZ=(P−OneHot)/N`；Linear Backward：`dX=dZW`、`dW=dZ^TX`。
4. 标准训练主 GEMM FLOPs 约 `6NHV`；重算型 Fused CE 上界约 `8NHV`。
5. 完整 Logits 显存为 `N×V×dtype_bytes`，真实例子中单份 BF16 Logits 可达 3.914 GiB。
6. Weight Tying 节省参数状态，但不解决动态 Logits 峰值。
7. Vocab TP 把 Weight、Logits、dW 分片，但增加全局 Softmax 统计和 `dHidden` 通信。
8. 真正的 Chunk/Fusion 必须控制 Autograd Saved Tensor 生命周期；普通 Python Loop 不保证峰值下降。
9. 优化应同时看显存、重算 FLOPs、GEMM 利用率、HBM、通信、数值误差与端到端 Step Time。
10. 动态网页可以先做解析探索，最终必须用 Memory Snapshot 和 Nsight 验证真实实现。

可选自测

1. Shape 与 FLOPs

若：

```
N=32768, H=4096, V=128000
```

请计算：

```
Logits Shape
BF16 Logits GiB
Forward Projection FLOPs
标准训练三个 GEMM FLOPs
```

2. 反向推导

从：

```
Z=XW^T
```

只根据 Shape 推导：

```
dX = ?
dW = ?
```

并指出各自收缩哪个轴。

3. 工程决策

如果你的优化把 LM Head 峰值从 8 GiB 降到 1 GiB，但 LM Head 时间增加 20%，你会检查哪些端到端指标来判断它是否值得上线?

继续学习

1. 前置：Stable Softmax、LogSumExp 与 Online Softmax —— 理解不物化全 Logits 的数学基础。
2. 同层对比：Megatron Vocab Parallel CE 与 CCE/Fused Linear CE —— 比较通信、重计算和 Tile 策略。
3. 下游应用：LM Head Kernel Profiling 与 Nsight 调优 —— 将显存模型落到真实 CUDA/Triton 时间线。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-27，本讲解基于稳定的矩阵微积分、Softmax Cross Entropy 原理，以及公开实现/资料：NVIDIA Megatron-LM `VocabParallelCrossEntropy` 源码、NVIDIA NeMo RL Chunked Linear Cross-Entropy 文档、Cut Cross-Entropy（Wijmans et al., ICLR 2025）与公开 Fused Linear Cross Entropy 实现说明。

不同框架对以下内容的实现可能不同：

```
是否保存 Logits 或 Softmax
是否原地覆盖
Backward 是否重算
Reduction dtype
Label Smoothing
Ignore Index
TP/SP/CP 支持
Chunk 维度
通信与计算重叠
```

文中的峰值是显式 Tensor 生命周期解析模型，不包含 Allocator 碎片和所有 Kernel Workspace；公开项目报告的显存/性能提升只适用于其测量配置，不能直接当作你的训练任务结果。